Peer Robotics Assessment

Computer Vision Intern

Drumil Kotecha
Under Mr. Vishrut Kaushik
April 26, 2026

1. **Look closely at the differences between the three images. In Image 1 and Image 2, the pallets are staged within blue tape lines. In Image 3, there are no blue lines. Define the exact visual heuristics your pipeline would use to confidently conclude "readiness to move" across all three distinct scenarios.**

Solution) My pipeline uses a prioritized cascade of visual checks. If the strongest signal is present, the robot acts on it. If it's absent, the pipeline falls through to secondary and tertiary cues.

Heuristic 1. Staging Zone Detection (Blue Tape Lines)

The camera looks for blue tape boundaries on the floor using color segmentation or a VLM prompt like "Are there blue tape lines on the floor near this pallet?" If blue lines are detected and a pallet is inside them, that's the strongest readiness signal, this pallet has already been placed here by a worker specifically to be picked up. In Image 1 and Image 2, this cue is available and sufficient.

Heuristic 2. Load Completeness and Wrapping State

If no blue lines are detected (Image 3 scenario), the pipeline falls to secondary cues. The camera assesses: Is the pallet fully loaded (no visible empty space on top, boxes/items stacked to a consistent height)? Is shrink wrap or stretch film visible around the load? Wrapping is a strong indicator that loading is complete. Workers don't wrap a pallet they're still adding to. In Image 3, the pallets are stacked and wrapped on top, so despite no blue lines, the pipeline can infer readiness from wrapping alone.

Heuristic 3. Human Activity and Workspace Context

The VLM checks if anyone is actively working near or on this pallet? Are there open boxes, loose items, or equipment nearby suggesting work in progress? If yes, mark this pallet as "not ready" regardless of other cues. If the area is clear of active work, confidence in readiness goes up.

Heuristic 4. Fork Accessibility and Pickup Feasibility

Before committing to a pickup, the camera checks: Are the fork pockets visible and unobstructed? Is there enough clearance on both sides to approach? Is the pallet sitting flat on the ground (not tilted, not stacked on another pallet in a way that prevents safe pickup)? The robot aligns itself to center on the pallet for balanced weight distribution.

Heuristic 5. Spatial Placement at Destination

When delivering, the robot uses its camera to scan the staging area. It detects existing pallets already placed within blue lines and estimates available space. It places the new pallet adjacent to existing ones in a systematic row/column layout, ensuring it stays within the blue boundaries and leaves enough clearance for the next pallet.

Applied to each image:

- **Image 1:** Blue tape visible, loaded pallet in staging zone → Heuristic 1 triggers → ready. Also shows an empty pallet on the left with no load → pipeline correctly skips it since Heuristic 2 (load completeness) fails.
- **Image 2:** Blue tape visible, multiple fully wrapped pallets inside lines → Heuristic 1 + 2 both confirm → high confidence ready.
- **Image 3:** No blue tape, but pallets are fully stacked and wrapped, no active workers visible → Heuristic 1 fails, falls to Heuristic 2 (wrapped = complete) + Heuristic 3 (no human activity) → pipeline still concludes ready, with slightly lower confidence score.

2. Beyond readiness, the AMR must physically engage the pallet. How would your pipeline confirm that the fork pockets (the dark voids at the base) are structurally clear of debris before driving into them?

Solution) My pipeline uses a prioritized cascade of visual checks. If the strongest signal is present, the robot acts on it. If it's absent, the pipeline falls through to secondary and tertiary cues.

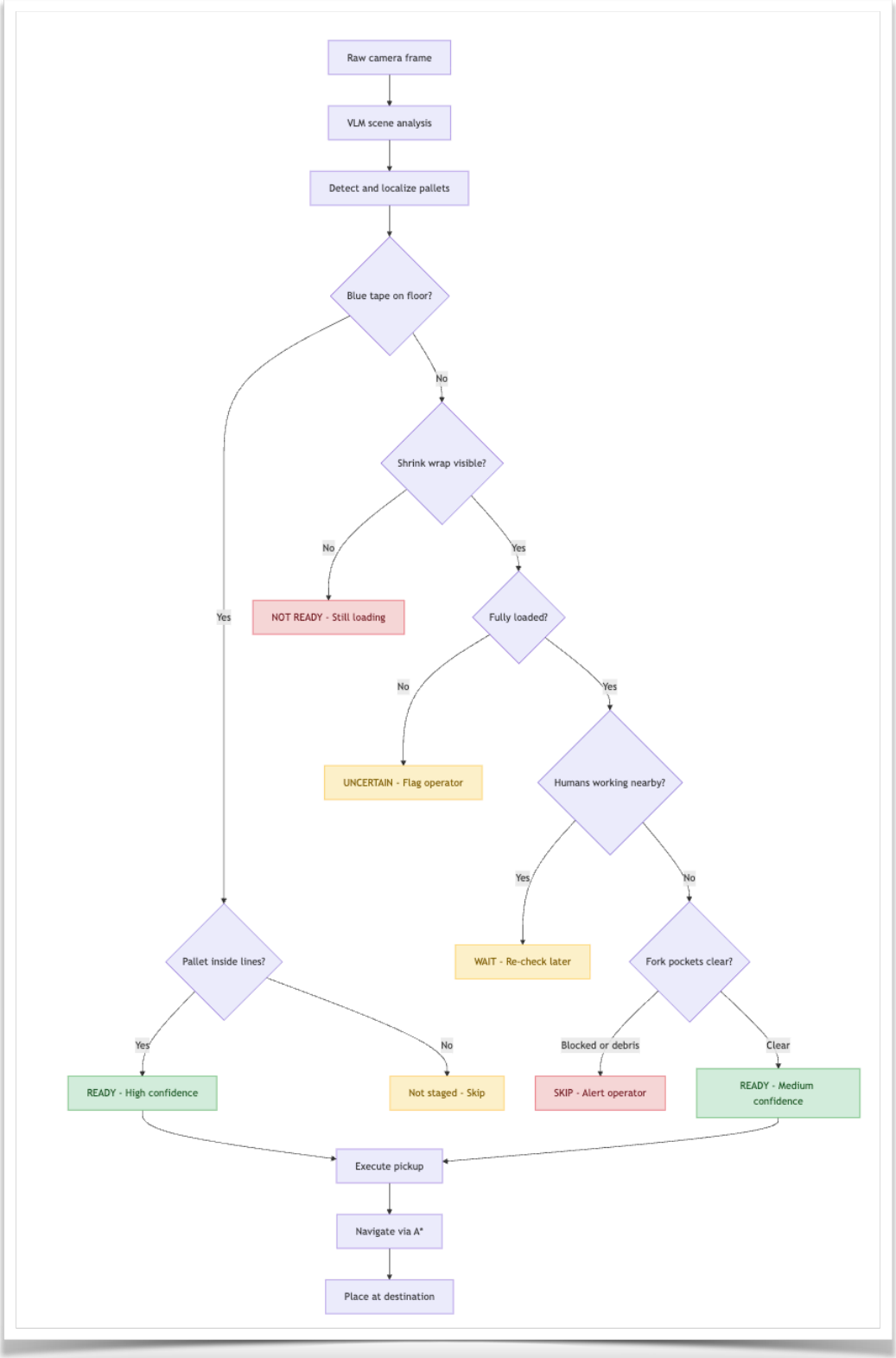
Geometry Check: The robot firstly analyzes the height and width of the pocket using depth estimation. We compare this against the fork size. If it crosses the threshold, we let the operation of insertion happen, else let's say the opening is too narrow or small, the robot skips and flags it.

Obstruction Check: Fork pockets should appear as dark, uniform voids when empty. The pipeline looks inside the pocket for irregularities, unexpected color variations, textures, or shapes that break the uniform darkness. Depth sensing helps: an empty pocket gives consistent depth readings across the opening, while debris causes uneven, shallower readings at certain points. A VLM prompt like "Is there any obstruction or debris visible inside the fork pocket?" adds a semantic layer of confirmation.

If debris is detected, the robot does not attempt insertion. It logs the issue with a camera snapshot, alerts a human operator, and moves on to the next available pallet.

3. **Write a step-by-step solution flow diagram from raw input image -> your solution -> output: "is the pallet ready to be moved?"**

Solution)



```
flowchart TD
    A[Raw camera frame] --> B[VLM scene analysis]
    B --> C[Detect and localize pallets]
    C --> D{Blue tape on floor?}

    D -->|Yes| E{Pallet inside lines?}
    E -->|Yes| F[READY - High confidence]
    E -->|No| G[Not staged - Skip]

    D -->|No| H{Shrink wrap visible?}
    H -->|No| J[NOT READY - Still loading]
    H -->|Yes| I{Fully loaded?}
    I -->|No| L[UNCERTAIN - Flag operator]
    I -->|Yes| K{Humans working nearby?}
    K -->|Yes| M[WAIT - Re-check later]
    K -->|No| N{Fork pockets clear?}
    N -->|Blocked or debris| P[SKIP - Alert operator]
    N -->|Clear| Q[READY - Medium confidence]

    F --> S[Execute pickup]
    Q --> S
    S --> T[Navigate via A*]
    T --> U[Place at destination]

    style F fill:#d4edda,stroke:#28a745,color:#155724
    style Q fill:#d4edda,stroke:#28a745,color:#155724
    style J fill:#f8d7da,stroke:#dc3545,color:#721c24
    style P fill:#f8d7da,stroke:#dc3545,color:#721c24
    style G fill:#fff3cd,stroke:#ffc107,color:#856404
    style M fill:#fff3cd,stroke:#ffc107,color:#856404
    style L fill:#fff3cd,stroke:#ffc107,color:#856404
```

Click here: Mermaid link

4. **Select an open-source VLM (e.g., PaliGemma, LLaVA, Moondream). Write the specific prompt you would use for this task, run the provided images through your chosen model, and attach the raw results/outputs.**

Solution) I selected LLaVA (Large Language and Vision Assistant), a 7B parameter open source VLM that pairs a CLIP ViT-L/14 vision encoder with a Vicuna language model connected through a linear projection layer. I ran it locally via Ollama on a MacBook Pro. I chose LLaVA because it runs without cloud dependencies (important for an AMR operating on premise), supports visual QA out of the box, and can be quantized for edge deployment on hardware like the Jetson AGX Orin.

Prompts

I designed four targeted prompts, each mapping to a specific heuristic in my readiness pipeline:

Prompt 1 (Scene Understanding): General environment description, pallet count, wrapping state, floor markings, human presence.

Prompt 2 (Readiness Classification): Binary READY/NOT READY classification per pallet with explicit criteria definitions provided in the prompt.

Prompt 3 (Fork Pocket Check): Focused inspection of pallet base for debris and obstruction.

Prompt 4 (Blue Tape Detection): Dedicated floor marking detection.

I split into four prompts rather than one because each maps to a specific heuristic in the pipeline, and focused prompts produce more reliable answers than a single overloaded query.

Raw Result: https://github.com/Drumil23/PR_Assessment/blob/master/pallet_readiness_results.json

Key Findings and Honest Error Analysis

Image 1: LLaVA partially failed. The scene understanding prompt triggered a refusal ("As an AI, I don't have the ability to physically be in a location"), which is a known LLaVA behavior where the role prompt gets overridden by its safety training. The fork pocket check worked correctly, identifying clear pockets. However, blue tape detection missed the blue floor markings that are visible in the image.

Image 2: Best performance of the three. Scene understanding correctly identified shrink wrapped pallets, blue tape lines on the floor, and no humans nearby. Readiness classification identified some pallets as READY with correct reasoning (fully loaded,

wrapped, fork pockets visible). However, it hallucinated a worker on one pallet that isn't actually there. Blue tape detection incorrectly said no blue tape exists, contradicting its own scene understanding response.

Image 3: Scene understanding detected plastic wrapping correctly but hallucinated blue tape lines on the floor (there are none in this image). Interestingly, the dedicated blue tape prompt correctly reported no blue lines, showing that focused prompts outperform general ones. Readiness classification was overly cautious and refused to commit to a clear READY/NOT READY decision.

Failure Modes Identified

a. Blue tape detection is unreliable. LLaVA missed it in Images 1 and 2 where it exists, and hallucinated it in Image 3's scene understanding. Fix: supplement with classical CV using HSV color segmentation in the blue range, which is deterministic and reliable.
b. Role prompt rejection. LLaVA sometimes ignores the "you are a vision system" framing and falls back to generic AI disclaimers. Fix: use shorter, more direct prompts without role playing, or switch to a model with less aggressive safety filtering like Moondream.
c. Hallucinated details. LLaVA invented a worker in Image 2 and described five pallets in Image 1 when fewer are visible. Fix: cross validate VLM outputs with object detection (YOLOv8) for ground truth pallet count and human detection.
d. Hedging on classification. For Image 3, the model refused to commit to a decision. In a production system this would be handled by the confidence scoring layer. Low confidence triggers an "UNCERTAIN" state and flags for operator review rather than blocking the pipeline.

5. Say the Jetson AGX Orin has only 5GB of available VRAM and 20% GPU, and you cannot run this modern VLM at an input of 1080p/30fps under these constraints. How will you architect and optimize your solution to achieve the task without crashing the system? Think of an innovative pipeline design that achieves the result without compromising the robot's real-time navigation.

Solution) No need to run the VLM on every frame. Divide work into levels.

Level 1 runs all the time at 30fps. It's a tiny object detector, YOLOv8-nano, quantized to INT8 using TensorRT. It uses less than 500MB of VRAM. All it does is spot pallets in the frame and draw bounding boxes. This keeps navigation running smoothly without any interruption.

Level 2 only activates when Level 1 finds a pallet. The robot slows down, captures one clean photo, crops just the pallet region (640x480 instead of full 1080p, which cuts memory usage by 75%), and sends that crop to a quantized VLM (Moondream or PaliGemma in INT4/INT8 via TensorRT or ONNX Runtime). The VLM runs the readiness prompts on that single crop. Since this happens once per pallet encounter, not 30 times a second, VRAM stays well within 5GB.

Level 3 is a classical CV fallback that runs entirely on CPU with zero VRAM. If the VLM is still processing, basic checks keep running: HSV color segmentation for blue tape, contour detection for fork pockets, and edge detection for wrap. These act as a safety net.

Other important decisions: VLM inference runs on a separate async thread so it never blocks navigation. Even Level 1 processes every 3rd to 5th frame since the robot moves slowly. The VLM model loads once at boot and stays in memory rather than loading and unloading. All models are compiled with TensorRT FP16/INT8 to squeeze maximum performance from the Jetson.

The result: navigation uses about 15% GPU through YOLO-nano. The VLM spikes to roughly 80% for about 500ms per pallet encounter, then drops back. Average VRAM stays under 3GB.

6. **Plastic wrap glare and harsh, varying warehouse lighting frequently break purely vision-based models. Look at the intense reflections in Image 2. If your VLM pipeline fails to reliably detect the pallet's boundaries or "readiness" due to this glare, what specific traditional CV technique (e.g., contour detection, specific filtering) or alternative sensor data would you fuse into your stack to prevent a failure or collision?**

Solution) Image 2 shows the problem. The shiny shrink wrap creates bright reflections that wash out the image, making it hard for any vision model to see pallet edges clearly.

Fix A: CLAHE preprocessing. Before any model sees the frame, apply CLAHE (a contrast normalization technique in OpenCV). It balances out the bright glare spots and dark shadows so the image becomes usable. One line of code, very fast.

Fix B: Polarizing filter on the camera. A physical filter on the lens that cuts glare from shiny surfaces like plastic wrap. Cheap hardware fix, big improvement for all downstream software.

Fix C: Add a depth camera. Glare is an RGB problem. Depth sensors like the Intel RealSense D455 don't care about reflections. They measure actual distance. So even if the camera image is washed out, the depth map still shows exactly where the pallet starts and the floor ends. Use depth for boundary detection and fork pocket inspection, use RGB as a backup.

Fix D: Ultrasonic sensors as a last resort. Cheap sensors that measure proximity using sound waves. Completely immune to lighting. If the robot is approaching a pallet and the camera is confused by glare, ultrasonic sensors still tell it "you are 15 cm away, slow down." If ultrasonic and camera disagree, the robot stops. Safety first.

7. **Your pipeline works perfectly for this specific warehouse. Next month, we deploy a fleet of 5 robots to a new facility that uses *yellow* lines, entirely different payloads on top of the pallets, and different lighting. How will you scale your solution beyond the initial facility without starting from scratch?**

Solution) Build for configuration, not retraining.

a. **Config files per warehouse.** Don't hard code "blue tape." Put the color range in a settings file. New warehouse uses yellow lines? Just change the color values in the config. A technician points the camera at the floor markers, system auto calibrates in five minutes.

b. **VLM prompts already generalize.** You don't retrain the model, you just change the question. "Are there yellow lines?" works the same as "Are there blue lines?" Different payloads like auto parts instead of boxes? The VLM can describe what it sees without needing to be taught.

c. **LoRA adapters for tricky cases.** If the VLM struggles at a specific site, collect 10 to 20 images and fine tune a tiny adapter (under 1MB) on top of the same base model. No full retraining.

d. **Lighting normalization.** Apply auto white balance and CLAHE before the model sees any frame. Makes every warehouse look consistent regardless of lighting type.

e. **Overall:** One shared model across all robots. Each facility just gets a config file and optional adapter pushed over the air.

8. **Before deploying this scaled solution to the live robots, how would you quantitatively evaluate your pipeline's performance? Define the specific metrics you would use to prove it is ready.**

Solution)

Detection accuracy:

Precision: Out of everything the system called "ready," how many were actually ready? If this is low, the robot keeps trying to pick up pallets that aren't ready, wasting time. Target: above 92%.

Recall: Out of all the pallets that were actually ready, how many did the system catch? If this is low, ready pallets get ignored and sit there. Target: above 95%.

F1 score: A single number that combines precision and recall. Target: above 93% before going live.

Confusion matrix: A table that shows exactly where the system gets confused. For example, does it keep calling unwrapped pallets "ready"? That tells you what to fix.

Safety metrics:

Fork pocket false positive rate: How often does the system say "pockets are clear" when there's actually debris inside? This causes real physical damage.
Target: below 0.5%.

Human detection recall: If a worker is near a pallet, does the system always catch it? Target: above 99%.

Collision rate: How many times does the robot bump into something per 1000 pickups? Target: 0.

Operational metrics:

Latency: Time from camera seeing the pallet to making a decision. Must stay under 500ms so the robot doesn't freeze mid navigation.

Throughput: Pallets moved per hour. This is the number the customer actually cares about. Compare it against a human forklift operator.

Uptime: How long does the system run without crashing or running out of memory on the Jetson.

Generalization score: Run the same tests at multiple warehouses. If accuracy drops more than 10% at a new site, the system is too dependent on the original warehouse.

To test, collect 250 plus labeled images per warehouse, including the edge cases like glare, debris, partial loads, and workers nearby. Have the actual warehouse operators label what's ready and what's not. Run the pipeline on that set, compute all the numbers above, and set a pass/fail line before any robot goes live.